

Absolutely. Below is a **complete beginner-to-interview-level guide** to **Relational Database Modeling & SQL Foundations**, with practical SQL examples, relationship diagrams, normalization, common mistakes, and interview questions.

Relational Database Modeling & SQL Foundations

1. Table, Row, Column, and Schema

Table

A **table** stores related data in a structured format.

Example: students

id	name	email	age
1	Ali	ali@gmail.com	22
2	Ahmed	ahmed@gmail.com	23
3	Muneeb	muneeb@gmail.com	21

Row

A **row** represents one complete record.

1 | Ali | ali@gmail.com | 22

This represents one student.

Column

A **column** represents one property/attribute.

id

name

email

age

Schema

A **schema** describes the structure of a database, including:

- Tables
- Columns
- Data types
- Primary keys
- Foreign keys
- Constraints
- Relationships

Example:

```
CREATE DATABASE school;
```

```
USE school;
```

```
CREATE TABLE students (  
    id INT,  
    name VARCHAR(100),  
    email VARCHAR(100),  
    age INT  
);
```

Here the schema of students contains four columns.

2. Primary Key

A **primary key** uniquely identifies every row in a table.

Example:

```
CREATE TABLE students (  
    id INT PRIMARY KEY,  
    name VARCHAR(100),  
    email VARCHAR(100),  
    age INT  
);
```

Insert data:

```
INSERT INTO students VALUES  
(1, 'Ali', 'ali@gmail.com', 22),  
(2, 'Ahmed', 'ahmed@gmail.com', 23),  
(3, 'Muneeb', 'muneeb@gmail.com', 21);
```

Now:

id = 1 → Ali

id = 2 → Ahmed

id = 3 → Muneeb

The id uniquely identifies each student.

Primary key rules

A primary key:

- Must be unique
- Cannot be NULL
- Identifies a row
- A table normally has one primary-key constraint
- Can consist of multiple columns → composite primary key

Example of invalid data

```
INSERT INTO students VALUES  
(1, 'Hassan', 'hassan@gmail.com', 25);
```

This fails because id = 1 already exists.

Auto-increment primary key

Common in MySQL:

```
CREATE TABLE students (  
    id INT PRIMARY KEY AUTO_INCREMENT,  
    name VARCHAR(100),  
    email VARCHAR(100)  
);
```

Now:

```
INSERT INTO students (name, email)  
VALUES ('Ali', 'ali@gmail.com');
```

```
INSERT INTO students (name, email)  
VALUES ('Ahmed', 'ahmed@gmail.com');
```

The database automatically generates:

1 → Ali

2 → Ahmed

3. Foreign Key

A foreign key creates a relationship between two tables.

Suppose we have:

customers

id

name

and:

orders

id

customer_id

amount

orders.customer_id refers to [customers.id](#).

```
CREATE TABLE customers (  
    id INT PRIMARY KEY AUTO_INCREMENT,  
    name VARCHAR(100) NOT NULL  
);
```

```
CREATE TABLE orders (  
    id INT PRIMARY KEY AUTO_INCREMENT,  
    customer_id INT NOT NULL,  
    amount DECIMAL(10,2),  
  
    FOREIGN KEY (customer_id)
```

```
REFERENCES customers(id)
);
Insert:
INSERT INTO customers (name)
VALUES ('Ali'), ('Ahmed');
Then:
INSERT INTO orders (customer_id, amount)
VALUES
(1, 5000),
(1, 2500),
(2, 3000);
```

Relationship:

Customer

|

| 1

|

| many

↓

Orders

Ali can have multiple orders.

4. Primary Key vs Foreign Key

This is a **very common interview question**.

Primary Key	Foreign Key
Uniquely identifies a row	Creates relationship between tables
Cannot be NULL	Can be NULL unless NOT NULL is specified
Unique	Can contain duplicate values
Usually exists in parent table	Usually references parent table
One primary-key constraint per table	Multiple foreign keys possible

Example:

customers

id ← PRIMARY KEY

orders

customer_id ← FOREIGN KEY

So:

[customers.id](#)

↑

|

orders.customer_id

Interview answer

A primary key uniquely identifies a record within its own table, while a foreign key references a key in another table to establish a relationship between tables.

5. UNIQUE Constraint

UNIQUE prevents duplicate values.

Example:

```
CREATE TABLE users (  
    id INT PRIMARY KEY AUTO_INCREMENT,  
    username VARCHAR(50) UNIQUE,  
    email VARCHAR(100) UNIQUE  
);
```

This is valid:

```
INSERT INTO users (username, email)  
VALUES ('ali', 'ali@gmail.com');
```

```
INSERT INTO users (username, email)  
VALUES ('ahmed', 'ahmed@gmail.com');
```

This is invalid:

```
INSERT INTO users (username, email)  
VALUES ('ali', 'other@gmail.com');
```

Because:

ali

already exists.

Primary Key vs UNIQUE

PRIMARY KEY



Unique + NOT NULL

UNIQUE prevents duplicates, but its NULL behavior depends on the database system.

6. NOT NULL Constraint

NOT NULL means a column must have a value.

```
CREATE TABLE employees (  
    id INT PRIMARY KEY,  
    name VARCHAR(100) NOT NULL,  
    salary DECIMAL(10,2)  
);
```

This is valid:

```
INSERT INTO employees  
VALUES (1, 'Ali', 50000);
```

This fails:

```
INSERT INTO employees  
VALUES (2, NULL, 60000);
```

Because name is NOT NULL.

7. One-to-One Relationship

In a **one-to-one relationship**, one record in table A is related to at most one record in table B.

Example:

Person

|

| 1

|

| 1

↓

Passport

A person has one passport.

SQL

```
CREATE TABLE persons (  
    id INT PRIMARY KEY AUTO_INCREMENT,  
    name VARCHAR(100) NOT NULL  
);
```

```
CREATE TABLE passports (  
    id INT PRIMARY KEY AUTO_INCREMENT,  
    passport_number VARCHAR(50) UNIQUE NOT NULL,  
    person_id INT UNIQUE NOT NULL,  
  
    FOREIGN KEY (person_id)  
        REFERENCES persons(id)  
);
```

The important part is:

person_id INT UNIQUE

Why?

Without UNIQUE:

Passport 1 → Person 1

Passport 2 → Person 1

Passport 3 → Person 1

would be possible.

With UNIQUE:

Person 1 → Passport 1

only one passport can reference that person.

Another example

User

|

| 1

↓

UserProfile

8. One-to-Many Relationship

This is one of the **most common database relationships**.

Example:

Customer

|

| 1

|

| many

↓

Orders

One customer can have many orders.

SQL

```
CREATE TABLE customers (  
    id INT PRIMARY KEY AUTO_INCREMENT,  
    name VARCHAR(100) NOT NULL  
);
```

```
CREATE TABLE orders (  
    id INT PRIMARY KEY AUTO_INCREMENT,  
    customer_id INT NOT NULL,  
    order_date DATE NOT NULL,  
  
    FOREIGN KEY (customer_id)  
        REFERENCES customers(id)  
);
```

Insert:

```
INSERT INTO customers (name)  
VALUES
```

('Ali'),

('Ahmed');

Orders:

INSERT INTO orders (customer_id, order_date)

VALUES

(1, '2026-08-01'),

(1, '2026-08-02'),

(1, '2026-08-03'),

(2, '2026-08-04');

Result:

Ali

├── Order 1

├── Order 2

└── Order 3

Ahmed

└── Order 4

Important rule

In a one-to-many relationship:

The foreign key goes on the “many” side.

Customer 1 ----- * Orders

↑

FK here

9. Many-to-Many Relationship

Example:

Students



Courses

One student can take many courses.

One course can have many students.

Student 1 → Course A

Student 1 → Course B

Student 2 → Course A

Student 2 → Course C

You cannot normally represent this directly with one foreign-key column.

We create a **join table**.

students

courses

student_courses

SQL

```
CREATE TABLE students (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  name VARCHAR(100) NOT NULL
```

);

```
CREATE TABLE courses (  
    id INT PRIMARY KEY AUTO_INCREMENT,  
    name VARCHAR(100) NOT NULL  
);
```

```
CREATE TABLE student_courses (  
    student_id INT NOT NULL,  
    course_id INT NOT NULL,  
  
    PRIMARY KEY (student_id, course_id),  
  
    FOREIGN KEY (student_id)  
        REFERENCES students(id),  
  
    FOREIGN KEY (course_id)  
        REFERENCES courses(id)  
);
```

Insert students

```
INSERT INTO students (name)  
VALUES  
( 'Ali' ),
```

('Ahmed'),
('Muneeb');

Insert courses

INSERT INTO courses (name)

VALUES

('Java'),
('Database'),
('Spring Boot');

Create relationships

INSERT INTO student_courses

VALUES

(1, 1),
(1, 2),
(2, 1),
(2, 3),
(3, 1),
(3, 2);

Now:

Ali

├─ Java

└─ Database

Ahmed

└─ Java

└─ Spring Boot

Muneeb

└─ Java

└─ Database

10. Why Do We Need a Join Table?

Bad design:

students

id | name | courses

1 | Ali | Java, Database, Spring

This creates problems.

Searching becomes difficult:

WHERE courses LIKE '%Java%'

You also cannot easily enforce that every course actually exists.

Better:

students

1 | Ali

courses

1 | Java

2 | Database

3 | Spring

student_courses

1 | 1

1 | 2

1 | 3

This is relational database design.

11. Normalization

Normalization is the process of organizing data to reduce:

- Duplicate data
- Data inconsistency
- Update problems
- Insert problems
- Delete problems

The goal is to design tables where each piece of information is stored appropriately.

12. First Normal Form — 1NF

A table follows **1NF** when:

- Each cell contains a single/atomic value
- There are no repeating groups
- Each row can be uniquely identified

Bad

students

id | name | courses

1 | Ali | Java, SQL, Spring

courses contains multiple values.

That's not atomic.

Better

students

id | name

1 | Ali

courses

id | name

1 | Java

2 | SQL

3 | Spring

student_courses

student_id | course_id

1		1
---	--	---

1		2
---	--	---

1		3
---	--	---

Now each cell contains one value.

13. Common Mistake: Comma-Separated Values

Avoid:

id	name	phone_numbers
----	------	---------------

1	Ali	0300..., 0311..., 0322...
---	-----	---------------------------

This is bad relational modeling.

Instead:

users

id	name
----	------

1	Ali
---	-----

user_phones

user_id	phone
---------	-------

1	0300...
---	---------

1	0311...
---	---------

1	0322...
---	---------

Now each phone is a separate record.

14. Second Normal Form — 2NF

2NF requires:

1. The table is already in 1NF.
2. Every non-key column depends on the **whole primary key**, not just part of it.

This matters especially with **composite primary keys**.

Consider:

student_courses

student_id | course_id | student_name

Primary key:

(student_id, course_id)

But:

student_name

depends only on:

student_id

It doesn't depend on the complete key.

Therefore, this violates 2NF.

Bad

student_id | course_id | student_name

1 | 10 | Ali

1 | 20 | Ali

2	10	Ahmed
---	----	-------

Ali is unnecessarily repeated.

Better

students

student_id	student_name
------------	--------------

1	Ali
---	-----

2	Ahmed
---	-------

student_courses

student_id	course_id
------------	-----------

1	10
---	----

1	20
---	----

2	10
---	----

Now student information is stored separately.

15. Third Normal Form — 3NF

3NF requires:

1. Table is in 2NF.
2. Non-key columns should depend directly on the primary key, not on another non-key column.

Bad example

employees

id | name | department_id | department_name

Here:

id → department_id

department_id → department_name

So:

id → department_name

indirectly.

department_name depends on department_id, not directly on the employee's id.

Better

employees

id | name | department_id

departments

id | name

Relationship:

employees.department_id

↓

[departments.id](#)

16. 1NF vs 2NF vs 3NF

Remember this simple way:

1NF

One cell = one value

Avoid:

Java, SQL, Spring

in one column.

2NF

Depend on the whole key

Mostly relevant when you have a composite key.

Avoid:

(student_id, course_id) → student_name

because student_name only depends on student_id.

3NF

Non-key columns should not depend on other non-key columns

Avoid:

employee_id → department_id → department_name

Instead, create a separate department table.

17. Business Relationship First

A very important professional principle is:

Design the database based on business relationships, not just the data you currently have.

Suppose you're building a Doctor Appointment System.

Ask:

Who are the entities?

Possible entities:

Doctor

Patient

Appointment

Speciality

Then ask:

How are they related?

Speciality 1 — * Doctors

Doctor 1 — * Appointments

Patient 1 — * Appointments

Then create tables.

18. Complete Doctor Appointment Database Example

Since this is a very common real-world backend project, let's design it properly.

Specialities

```
CREATE TABLE specialities (  
    id INT PRIMARY KEY AUTO_INCREMENT,
```

```
    name VARCHAR(100) NOT NULL UNIQUE  
);
```

Doctors

```
CREATE TABLE doctors (  
    id INT PRIMARY KEY AUTO_INCREMENT,  
    name VARCHAR(100) NOT NULL,  
    email VARCHAR(150) NOT NULL UNIQUE,  
    speciality_id INT NOT NULL,  
  
    FOREIGN KEY (speciality_id)  
        REFERENCES specialities(id)  
);
```

Patients

```
CREATE TABLE patients (  
    id INT PRIMARY KEY AUTO_INCREMENT,  
    name VARCHAR(100) NOT NULL,  
    email VARCHAR(150) NOT NULL UNIQUE  
);
```

Appointments

```
CREATE TABLE appointments (  
    id INT PRIMARY KEY AUTO_INCREMENT,  
    doctor_id INT NOT NULL,
```

```
patient_id INT NOT NULL,  
appointment_date DATE NOT NULL,  
appointment_time TIME NOT NULL,
```

```
FOREIGN KEY (doctor_id)  
REFERENCES doctors(id),
```

```
FOREIGN KEY (patient_id)  
REFERENCES patients(id)
```

```
);
```

Relationship:

Speciality

|

| 1

| *

Doctor

|

| 1

| *

Appointment

*

|

| 1

Patient

This is a much better design than storing everything in one table.

19. Query the Relationships

Suppose we want to see appointments with doctor and patient names.

SELECT

[a.id](#),

[d.name](#) AS doctor_name,

[p.name](#) AS patient_name,

a.appointment_date,

a.appointment_time

FROM appointments a

JOIN doctors d

ON [a.doctor_id](#) = [d.id](#)

JOIN patients p

ON a.patient_id = [p.id](#);

Example result:

id	doctor_name	patient_name	appointment_date
----	-------------	--------------	------------------

1	Dr. Ahmed	Ali	2026-08-10
---	-----------	-----	------------

2	Dr. Sara	Muneeb	2026-08-11
---	----------	--------	------------

20. Complete SQL Example

Here is a small complete database you can actually run in MySQL.

```
CREATE DATABASE university;
```

```
USE university;
```

```
CREATE TABLE students (  
    id INT PRIMARY KEY AUTO_INCREMENT,  
    name VARCHAR(100) NOT NULL,  
    email VARCHAR(150) NOT NULL UNIQUE  
);
```

```
CREATE TABLE courses (  
    id INT PRIMARY KEY AUTO_INCREMENT,  
    name VARCHAR(100) NOT NULL UNIQUE  
);
```

```
CREATE TABLE student_courses (  
    student_id INT NOT NULL,  
    course_id INT NOT NULL,  
  
    PRIMARY KEY (student_id, course_id),
```

```
FOREIGN KEY (student_id)
REFERENCES students(id),
```

```
FOREIGN KEY (course_id)
REFERENCES courses(id)
```

```
);
```

```
INSERT INTO students (name, email)
```

```
VALUES
```

```
('Ali', 'ali@gmail.com'),
```

```
('Ahmed', 'ahmed@gmail.com'),
```

```
('Muneeb', 'muneeb@gmail.com');
```

```
INSERT INTO courses (name)
```

```
VALUES
```

```
('Java'),
```

```
('Database'),
```

```
('Spring Boot');
```

```
INSERT INTO student_courses (student_id, course_id)
```

```
VALUES
```

```
(1, 1),
```

```
(1, 2),
```

(2, 1),

(2, 3),

(3, 1),

(3, 2);

SELECT * FROM students;

SELECT * FROM courses;

SELECT * FROM student_courses;

To get students with their courses:

SELECT

[s.name](#) AS student,

[c.name](#) AS course

FROM student_courses sc

JOIN students s

ON sc.student_id = [s.id](#)

JOIN courses c

ON sc.course_id = [c.id](#);

Result:

student | course

-----|-----

Ali | Java

Ali | Database

Ahmed | Java

Ahmed | Spring Boot

Muneeb | Java

Muneeb | Database

21. Practical Interview Questions

Q1. What is a database table?

Answer:

A table is a structured collection of related data organized into rows and columns.

Q2. What is a row?

Answer:

A row represents one complete record in a table.

Q3. What is a column?

Answer:

A column represents an attribute or property of the data.

Q4. What is a primary key?

Answer:

A primary key uniquely identifies each record in a table. It cannot contain NULL values.

Q5. What is a foreign key?

Answer:

A foreign key is a column or group of columns that references a key in another table and establishes a relationship between tables.

Q6. Can a foreign key contain duplicate values?

Answer:

Yes.

For example:

customer_id

1

1

1

2

2

Multiple orders can belong to the same customer.

Q7. Can a primary key contain duplicate values?

Answer:

No. Primary key values must be unique.

Q8. What is the difference between PRIMARY KEY and UNIQUE?

Answer:

Both enforce uniqueness, but a primary key identifies the row and cannot be NULL. A UNIQUE constraint is used to prevent duplicate values in a column or combination of columns.

Q9. What is a one-to-many relationship?

Answer:

One record in one table can be associated with many records in another table.

Example:

Customer → Orders

One customer can have many orders.

Q10. Where is the foreign key placed in one-to-many?

Answer:

On the **many side**.

Customer 1 ——— * Order

Order.customer_id → [Customer.id](#)

Q11. How do you implement many-to-many?

Answer:

Using a third table called a **join table** or associative table.

Example:

students

courses

student_courses

Q12. Why can't we simply put course IDs in a student column?

Bad:

student_id | course_ids

1 | 1,2,3

Because it violates atomicity and makes searching, updating, indexing, and enforcing relationships difficult.

Q13. What is normalization?

Answer:

Normalization is the process of organizing database tables to reduce unnecessary duplication and prevent data anomalies.

Q14. What is 1NF?

Answer:

1NF requires each column to contain atomic values and prevents repeating groups.

Q15. What is 2NF?

Answer:

2NF means the table is in 1NF and every non-key attribute depends on the entire primary key.

Q16. What is 3NF?

Answer:

3NF means the table is in 2NF and non-key attributes do not depend on other non-key attributes.

Q17. Why is normalization important?

It helps prevent:

Update anomaly

Insert anomaly

Delete anomaly

and reduces unnecessary data duplication.

22. Interview Scenario

Interviewer:

Design a database for an e-commerce application.

Think:

User

Product

Order

OrderItem

Category

Relationships:

User 1 — * Order

Order 1 — * OrderItem

Product 1 — * OrderItem

Category 1 — * Product

Possible schema:

users

id PK

name

email UNIQUE

categories

id PK

name UNIQUE

products

id PK

name

price

category_id FK

orders

id PK

user_id FK

order_date

order_items

order_id FK

product_id FK

quantity

price

order_items acts as the join/association table between orders and products.

This kind of thinking is what interviewers usually want when they ask you to **design a relational database**.

23. Key Points to Remember

Database basics

Database



Schema



Tables



Rows + Columns

Primary key

Uniquely identifies a row

Foreign key

Creates relationship between tables

One-to-one

1 — 1

Example:

Person → Passport

One-to-many

1 — *

Example:

Customer → Orders

Foreign key goes on the **many side**.

Many-to-many

—

Use a join table:

Student

Course

StudentCourse

1NF

One cell = one value

2NF

Depend on the entire key

3NF

No non-key → non-key dependency

Most important interview distinction

PRIMARY KEY



Who is this row?

FOREIGN KEY



Which row in another table is this related to?

Most important design rule

First understand the business entities and their relationships, then design the tables.

That mindset will help you far more than memorizing normalization definitions.